RelaxedUnspecConstraint#

https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic_sha

Description:

For clients: no explicit constraint; constraint is whatever is implicitly inferred by guards from tracing. For backends: there must exist at least TWO possible values for the size at this dimension which satisfy the guards for this dimension. In other words, this constraint helps us distinguish between “we don’t care if this dimension specializes or not” versus “this dimension must be unspecialized.” However, this constraint doesn’t say very much about what specialization is permitted; for example, if we guard on a size being even, this would still be acceptable under an unspec constraint. This makes RelaxedUnspecConstraint useful for eager mode, where your backend compiler may add constraints to otherwise dynamic dimensions; we can’t assert that there are NO guards as this is brittle because compilers should be able to add extra constraints. If you want to assert that there are no guards, use StrictMinMaxConstraint with an unbounded ValueRanges.

Function Signature

```
class
torch.fx.experimental.symbolic_shapes.RelaxedUnspecConstrain
t(warn_only)[source]#
```

Detailed Documentation

Documentation

For clients: no explicit constraint; constraint is whatever is implicitly inferred by guards from tracing.

For backends: there must exist at least TWO possible values for the size at this dimension which satisfy the guards for this dimension.

In other words, this constraint helps us distinguish between “we don’t care if this dimension specializes or not” versus “this dimension must be unspecialized.” However, this constraint doesn’t say very much about what specialization is permitted; for example, if we guard on a size being even, this would still be acceptable under an unspec constraint. This makes RelaxedUnspecConstraint useful for eager mode, where your backend compiler may add constraints to otherwise dynamic dimensions; we can’t assert that there are NO guards as this is brittle because compilers should be able to add extra constraints. If you want to assert that there are no guards, use StrictMinMaxConstraint with an unbounded ValueRanges.